My Claude Code Setup: Auto Mode by Default

Why My Guardrails Aren't Permission Prompts

Idaliya Grigoryeva

UC San Diego

July 2026

Agentic Tools Session · UCSD Economics

What I'll Cover

- I'm the **non-expert** here: no terminal, no sandbox, no cluster, nothing at the OS level
- Everything runs in the **Claude desktop app, Code mode**, straight into my real project folders
- My permission mode is **Auto**, and the safety sits somewhere other than the permission prompt

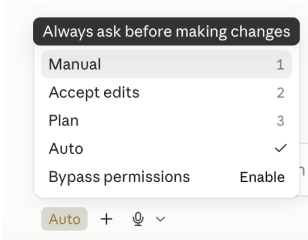
Four things, then questions

1. The modes, and why I only use Auto
2. My setup in one slide
3. What actually keeps me safe
4. What surprised me, and it wasn't permissions

The Four Modes, and the One I Use

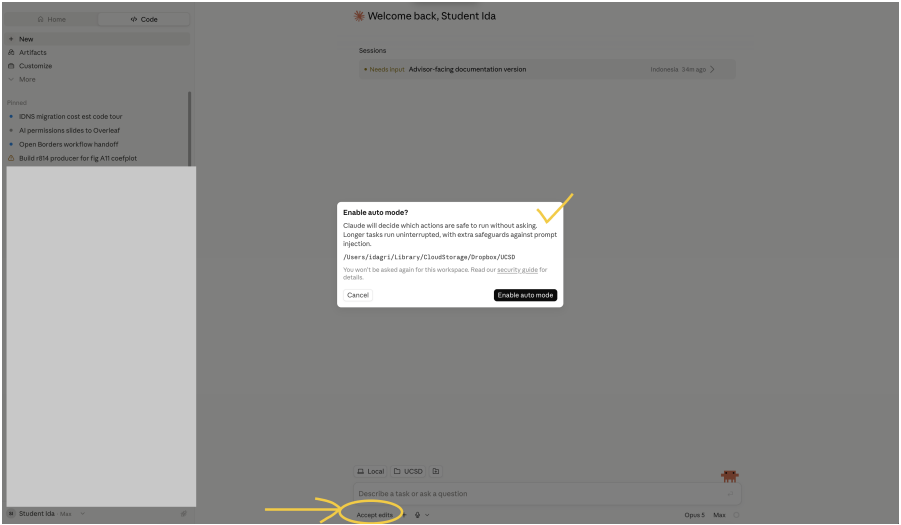
Mode	What it does	Me
Manual	Asks before every change	Never
Accept edits	Edits yes, commands ask	By accident, early on
Plan	Read-only, no changes	Never
Auto	Decides what's safe to run	**Always**
Bypass	Skips everything	Off, and stays off

I ran **Accept edits** for a while without realizing it, and it was **extremely annoying**: it still stopped at every command, which is most of what a real run does



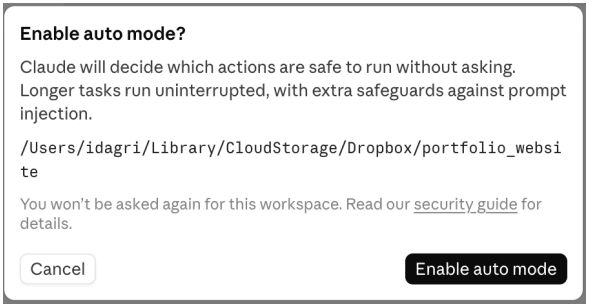
Bottom-left of the Code tab

Where the Permission Controls Live



Two places only: the **mode chip** at the bottom-left, and the **one-time dialog** when you first switch a folder to Auto

What Auto Actually Is



- It is **not** the same as bypassing permissions, which I leave off
- Claude still judges each action, and destructive or irreversible ones still surface
- It is set **per workspace**, so one folder at a time, not a global switch

Why I default to it

The prompts I was actually clicking through were `Read`, `Grep`, `python3`, never the ones that could hurt me. By the 30th prompt I was approving without reading, so the prompts had stopped buying safety and were only buying interruptions

My Setup, in One Slide

What I run

- Claude **desktop app**, **Code mode**, no terminal
- Max plan: Opus and Fable, 1M context
- Straight into my **real project folders**, Dropbox-backed
- One project per folder, with its data and its code

What I deliberately don't have

- No sandbox
- Network **on**
- Liberal permissions
- No identified or private data in *any* folder Claude can see

The condition that makes this reasonable

Manual review is worth its cost exactly when **I can't roll back**. With git, Dropbox history, and nothing secret in the folder, that is almost never. **Change that condition and my answer changes too**

The Rule That Does the Work: Never Delete

One rule in my global profile carries more weight than every permission prompt I turned off.

Rule (1), from my global profile

Do NOT delete anything that already exists when you work, only create new files. If you created something that is no longer relevant, **move it to a subfolder** `_archive_claude` that you can create in your working directory, **but do not delete**

Set once, in the desktop app, so it applies across **every** project on top of each project's own `CLAUDE.md`

Two more, and that's the whole net

- **Rollback**: git plus Dropbox version history, so a bad run gets reverted, not debugged
- **Written guidance**: a `CLAUDE.md` per project and a running lessons-learned file

Permissions are loose **because** these are in place, not instead of them

Cutting the Last Prompts: `settings.json`

`.claude/settings.json`, per project

```
"permissions": {
  "allow": [
    "Bash(python3:*)",
    "Bash(mkdir *)",
    "Bash(find *)",
    "Bash(curl *)",
    "Bash(unzip *)",
    "Write(**)",
    "Edit(**)"
  ]
}
```

- Pre-approves the handful of things **every** run touches, so there is almost nothing left to click even outside Auto
- Add your **Stata binary path** the same way if you run do-files
- Stata in **batch mode** writing a log means Claude reads the output itself instead of asking you to paste it

Most of my remaining prompts weren't a permissions problem, they were a **workflow** problem

What Surprised Me, and It Wasn't Permissions

- **Traffic goes out, and it adds up.** OCR-ing images and downloading datasets pull real bandwidth. On a **hotspot** that is your data plan, not just a slow spinner
- **Retries cost you twice.** A step that fails and retries burns the tokens again, and you can hit the credit ceiling mid-task with nothing to show for the run
- **A closed lid stops the run.** `caffeinate` keeps the Mac awake with the lid shut, which matters most when you are **moving between places** and want the job to survive the trip
- **So: make partial progress retrievable.** Have subagents **write output to disk as they go**, so a run that dies at 80% still leaves you the 80%

Same instinct as the never-delete rule

Assume the run breaks somewhere, and make the breakage **cheap**. Worth writing into `CLAUDE.md` **before** you need it, because you find out you needed it exactly when you can no longer do anything about it

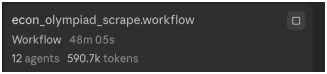
Two Features Worth Knowing

There is no live usage meter

- Claude **cannot** track its own token or traffic usage while it works, there is no running counter it can check mid-task
- But it **can estimate up front**: ask roughly how much this will take before kicking off a long job
- Cheap habit, and about the only budget signal you get

Background tasks vs. spin-off chats

- **Background tasks** run alongside what you are doing and report back into the same session
- **Spin-off chats** are a separate thread with their own context, for work that shouldn't share the current one



Still going at 48 minutes

Where This Would Break

- **I have not solved privacy-sensitive data.** No identified records sit in a folder Claude can see, so my folders hold **nothing secret and nothing irreplaceable**
- That is the actual precondition for everything on the previous slides, not a footnote to them
- **Recoverable is not the same as not.** A deleted file is bad but fixable. Leaked PII or a published API key is neither, and no amount of rollback helps
- **Read** permissions matter more than they look: with prompt injection, "it can read this" becomes "it can act on this"

What would make me tighten up

PII in the project folder · credentials or API keys in the repo · anything that can **send or publish on my behalf**. At that point I would want a sandbox, not my setup

Questions?

Happy to share the settings file or `CLAUDE.md` excerpts with anyone who wants them

These slides

idagri.github.io/files/ai-permissions-workshop-slides.pdf

My global `CLAUDE.md`, all nine rules

idagri.github.io/files/ida-claude-md.md

Idaliya Grigoryeva · idaliya.gri@gmail.com